

Projektablauf

Domain-Analyse und Datenbeschaffung

Deadlock, das Spiel, das sich im Zuge dieses Projektes angeschaut wird, ist ein **Third Person Shooter** und ein **Multiplayer Online Battle Arena (MOBA)**-Spiel, das von Valve entwickelt wird.

Zwei Teams mit je sechs Spielern kämpfen auf drei Lanes, um mit ihren NPC-Truppen ("Troopers") zum gegnerischen "Patron" (Basis) vorzudringen. Unterwegs müssen Wächter und Walker, die auf den einzelnen Lanes stehen und den Patron beschützen, zerstört werden.

Mit "Souls" bzw. auch eher allgemein bekannt als "Gold" kauft man Items um den Schaden, die Verteidigung oder Heilung des kontrollierten Charakters zu verbessern, und sich somit einen Vorteil gegenüber dem gegnerischen Team zu verschaffen.

Jeder spielbare Charakter hat unterschiedliche Fähigkeiten, Stärken und Schwächen (sogenannte "Heroes", manche sind stärker früh im Spiel, manche später). Aktuell gibt es 32 spielbare Charaktere.

Da sich das Spiel noch in der Alpha befindet, bietet Valve selbst keine öffentliche API an. Es gibt jedoch ein Community-Projekt namens "Deadlock API" (<https://deadlock-api.com/>), bei welchem Matchdaten automatisch gesammelt und öffentlich bereitgestellt werden. In diesem Projekt wurden die öffentlichen Datenbankdumps der Deadlock API als Datenquelle verwendet.

Datenaufbereitung

Datenaufbereitung & Feature Engineering

Für unser Modell sind einige Daten nicht relevant, die aber dennoch in dem Dump drinstehen. Diese Daten werden in einem ersten Schritt entfernt, um die Datenmenge zu reduzieren und die Verarbeitung zu beschleunigen. Dazu gehören Informationen wie z.B. welche Items die Spieler gekauft haben, oder wie viele Kills/Deaths/Assists sie hatten (und zu welchem Zeitpunkt im Spiel).

Es werden sich nur Spiele innerhalb eines Patches (25.10.2025 - 21.11.2025) angeschaut, da sich die Spielmechaniken und die Balance des Spiels u.A. durch Updates ändern können. Beispielsweise werden Helden generiert (abgeschwächt) oder gebufft (verstärkt), was die Winrates beeinflussen kann.

Noch dazu werden nur Spiele mit einem bestimmten (hohen) Skilllevel (ca. Top 13.4%) betrachtet, um die Varianz in den Daten zu reduzieren. Spiele mit Spielern, die das Spiel innerhalb der Spielzeit verlassen haben, werden ebenfalls entfernt. Genauso wie Spiele mit Bots, Cheatern, oder private Spiele.

Es werden nur Spiele betrachtet, die mindestens 8 Minuten gedauert haben (ein normales Match geht ca. 20-30 Minuten), in denen 2 Teams gegeneinander antreten, deren durchschnittlicher Rang sich um höchstens 2 Stufen unterscheidet (z.B. Ascendant 3 vs Ascendant 5).

Explorative Analyse

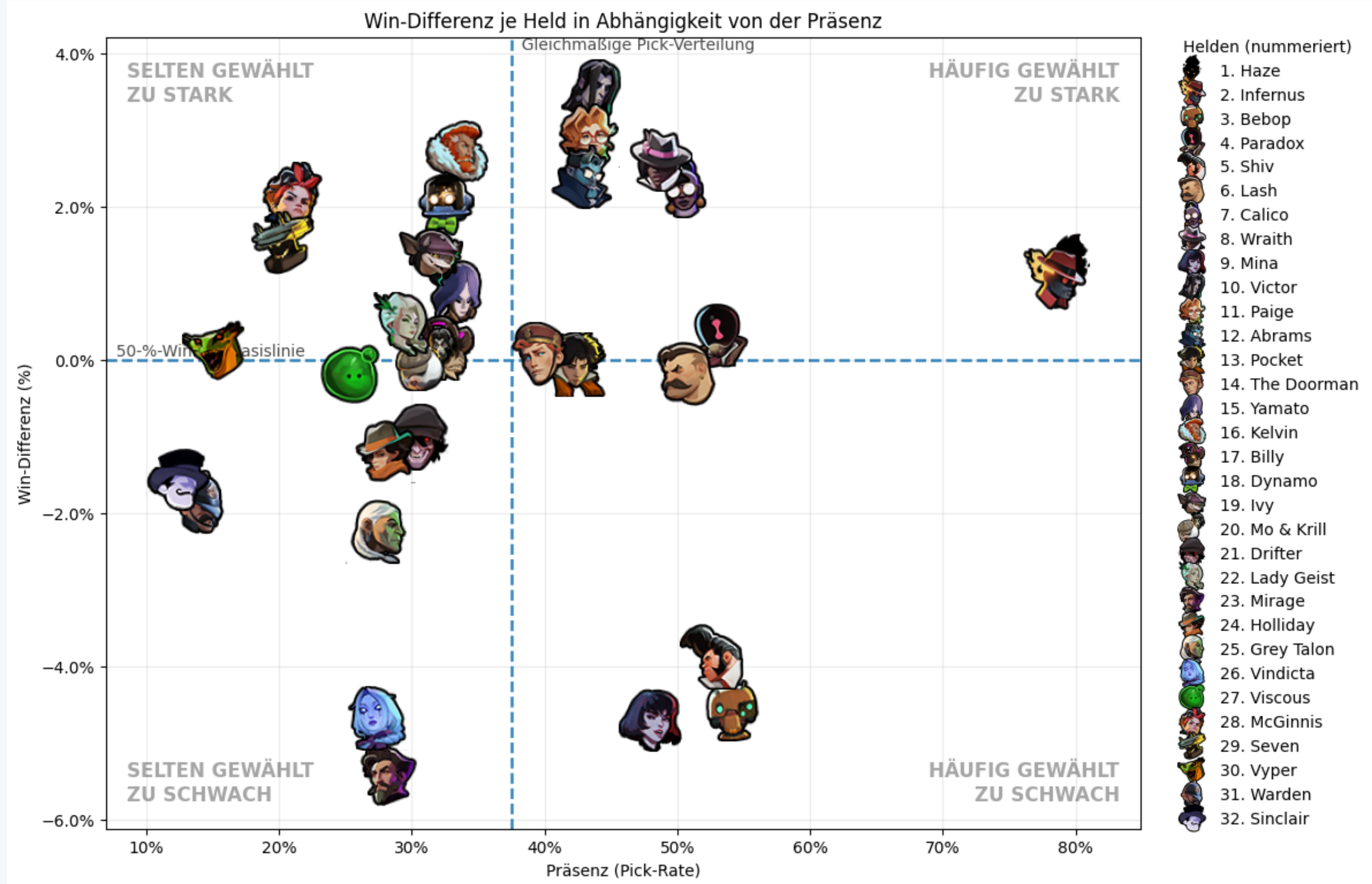


Abbildung 1. Winrate vs Pickrate der Helden

Wie man an den vorangegangenen Grafiken erkennen kann, bedeutet eine hohe Pickrate eines Helden nicht zwingend eine hohe Winrate. Es gibt Helden, die sehr oft gepickt werden, aber eine niedrige Winrate haben (z.B. Bebop).

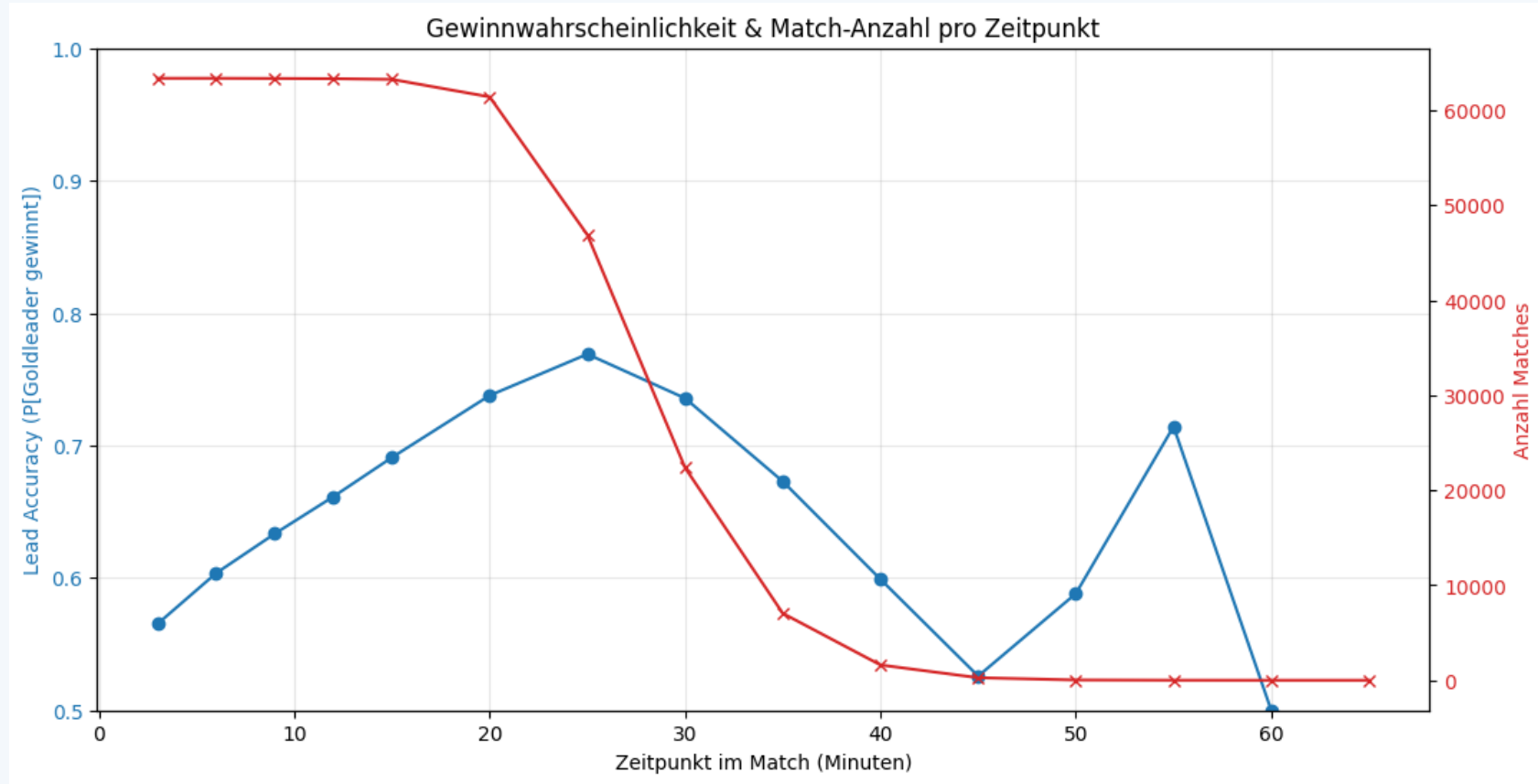


Abbildung 2. Goldvorsprung und Matchanzahl

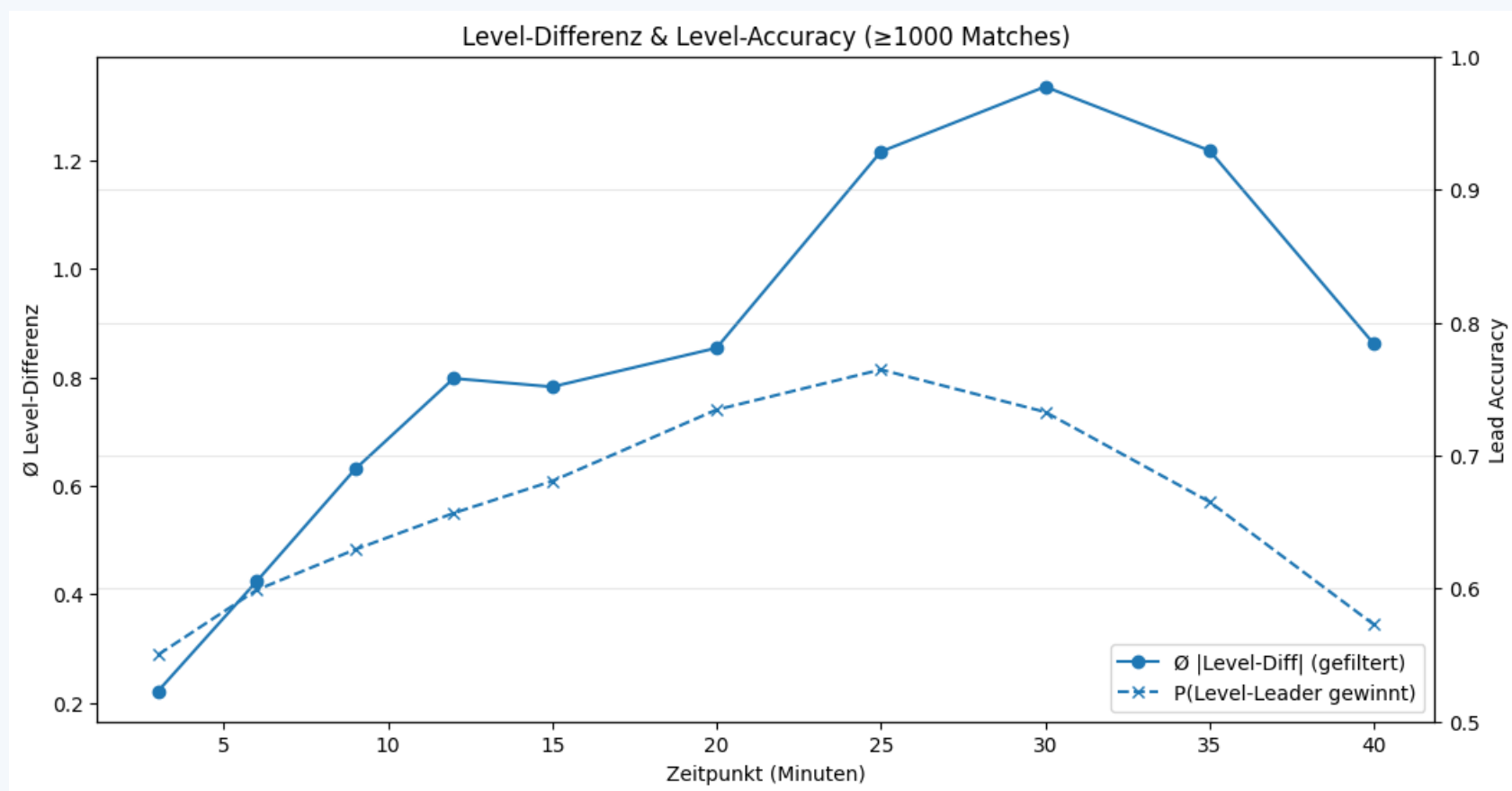


Abbildung 3. Leveldifferenz im Zeitverlauf

Feature Engineering

Zur maschinellen Verarbeitung im Neuronalen Netz haben wir nun im weiteren unsere Daten One-Hot-Encoded. Dies betraf im besonderen die Namen der Helden und die Objectives der jeweiligen Teams (ob die zu einem Zeitpunkt des Spiels noch standen oder nicht). Weiterhin haben wir die unsere Datensätze zu einem zusammengefasst um den Vorgang zu simplifizieren.

Wir berechnen die Gesamtmenge an Gold, die im Umlauf ist, und speichern diese. Zusätzlich ersetzen wir den absoluten Goldwert jedes Spielers durch seinen Anteil an dem gesamten Gold. Dadurch kann das Modell leichter erkennen, welche Spieler gerade besonders viel Gold im Vergleich zu anderen haben, ohne dass späte Spielphasen (mit höheren absoluten Goldmengen) vom Modell übergewichtet werden.

Zudem wenden wir z-Standardisierung (Standard Scaling) auf die numerischen Features an, um diese zu normalisieren. Die Parameter für die Normalisierung werden gespeichert, damit diese später auch auf neue Daten angewendet werden können.

Modell Training

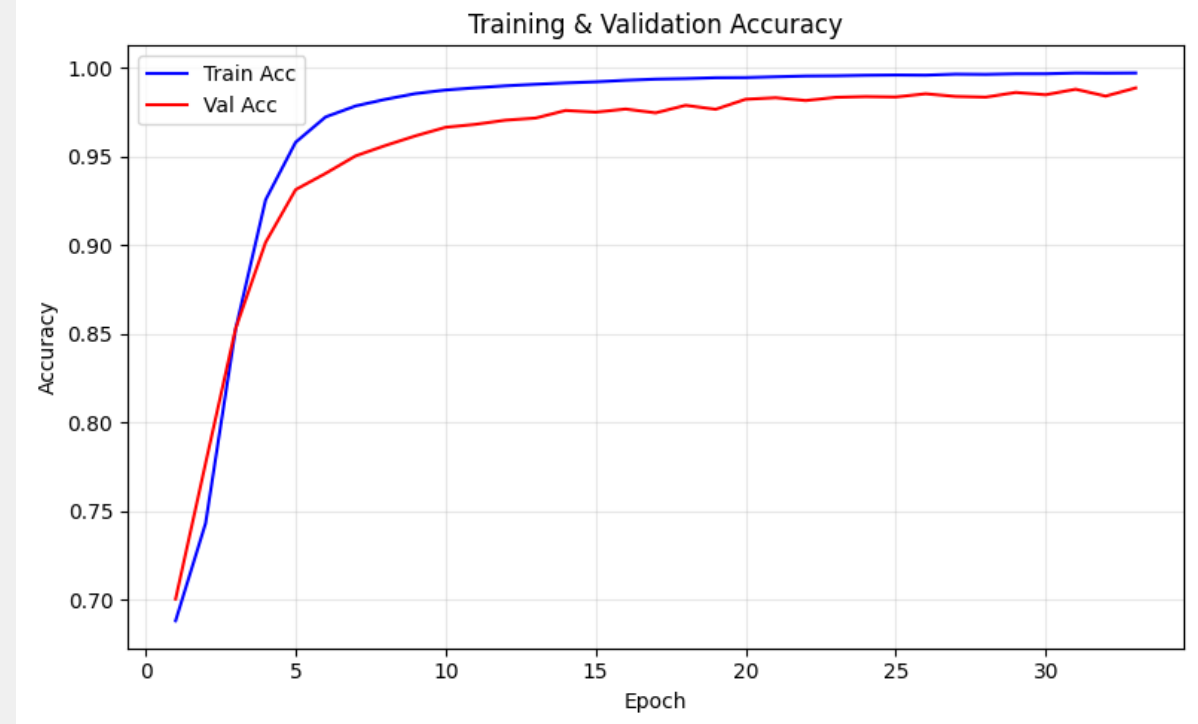
Unser Modell umfasst ein Fully-Connected Neuronales Netzwerk, welches mit PyTorch implementiert wurde. Die Anzahl der Hidden Layers und Neuronen pro Layer ist frei anpassbar. Als **Aktivierungsfunktion** wurde ReLU verwendet, um Nicht-Linearitäten im Modell abzubilden. Als **Loss-Funktion** wurde Binary Cross-Entropy Loss (mit Logits für verbesserte numerische Stabilität) verwendet, da es sich um ein Klassifikationsproblem handelt. Als **Optimierer** wurde Adam verwendet, da dieser adaptiv die Lernrate anpasst und in der Praxis oft gute Ergebnisse liefert. Wir haben uns als **Regularisierungsmethode** für Early Stopping entschieden, um Overfitting zu vermeiden. Das Training wird abgebrochen, wenn der Validierungsloss über 5 Epochen nicht um mindestens 1% verringert werden konnte. Die maximale Anzahl an Epochen wurde auf 200 gesetzt, aber aufgrund des Early Stop-pings wird nicht die volle Anzahl an Epochen durchlaufen (maximal ca. 60 Epochen).

Um die Hyperparameter zu tunen (Learning Rate, Anzahl Neuronen pro Layer, Anzahl Hidden Layers) wurde eine Grid Search durchgeführt, und die Parameterkombination mit dem niedrigsten Validierungsloss ausgewählt.

Ergebnisse



(a) Trainings und Validierungsloss



(b) Trainings und Validierungsaccuracy

Abbildung 4. Trainingsverlauf des Modells.

In den beiden Grafiken sieht man, dass der Trainings- und Validierungsloss sowie die zugehörige Accuracy über die Epochen hinweg stetig besser werden (Accuracy steigt, Loss sinkt). Für ein paar Epochen verschlechtert sich das Modell, jedoch sieht man in den Grafiken, dass insgesamt nach den Epochen wieder eine Verbesserung eintritt.

Evaluation

Modell	Accuracy	BCE	MSE
Dummy (Most Frequent)	50%	18,007	0,5
Dummy (Random)	50%	0,693	0,25
Logistic Regression	68,88%	0,575	0,197
Random Forest	67,86%	0,593	0,204
Neural Network	98,48%	0,052	0,012

Tabelle 1. Modellvergleich auf dem Testdatensatz. Accuracy: höher ist besser; BCE, MSE: niedriger ist besser.

Für eine abschließende Bewertung wurden das neuronale Netz (mit den Hyperparametern aus der Grid Search), sowie einige einfache Klassifikatoren (Dummies, Logistische Regression, Random Forest) trainiert bzw. berechnet und Accuracy, BCE und MSE auf den Klassifikatoren unbekannten Testdaten gemessen. Dies wurde 25-mal mit zufälligen, aber für alle Klassifikatoren gleichen Train-Test-Splits wiederholt und für die Messwerte jeweils der Durchschnitt berechnet (siehe Tabelle 1). Es ist zu sehen, dass das trainierte neuronale Netz weitaus aussagekräftiger ist als die anderen Klassifikatoren.

Reflektion und Ausblick

Reflektion

- Datenvorbereitung hat deutlich mehr Zeit in Anspruch genommen als erwartet.
- Aktuell ist das Modell nur für Matches mit hohem Skilllevel trainiert und somit nur für diese Matches akkurat aussagekräftig.
- Das Modell ist um einiges besser als erwartet : Als Ergebnis erhalten wir eine Accuracy von 98.48%, was etwas überraschend ist, da vor allem bei frühen Spielphasen (bis 10 Minuten) der Gewinner noch sehr schwer vorherzusagen ist.

Ausblick

- Erweiterung des Datensatzes um weitere Skilllevels.
- UI für Eingabe eines Gamestates der für die Prediction genutzt werden kann.
- Einfügen von weiteren Parametern in das Modell wie Items, KDA, Todeszeitpunkt etc.

Datengrundlage und Projekt-Repository

Datenquelle: <https://deadlock-api.com/>

Erhobene Daten: 58867 Spiele vom Zeitraum 25.10.2025 - 21.11.2025

